L01 - 什么是大语言模型

> **"大道至简，从文字接龙说起"**

📌 本节目标

1. 理解语言模型的核心思想——"预测下一个词"
2. 知道什么是"大"语言模型，以及参数量意味着什么
3. 了解 GPT 系列的发展脉络和 MiniMind 的定位
4. 掌握 Next Token Prediction、Temperature、采样策略等基本概念
5. 理解 Decoder-Only 架构为何成为主流

📚 前置知识

- 会用电脑，知道什么是"文件""程序"
- 对"人工智能""ChatGPT"有基本印象（哪怕只是听说过）
- 不需要任何编程或数学基础

正文讲解

1. 什么是语言模型？——从文字接龙说起

还记得小时候玩的**文字接龙**游戏吗？

> "天" → "天空" → "天空很蓝" → "天空很蓝，白云飘飘"

你的大脑在做一件事：**根据已有的文字，猜下一个最合适的词**。

语言模型（Language Model, LM） 做的事情和你一模一样！它是一个数学程序，输入一段文字，输出"下一个词最可能是什么"。

举个例子：

```
输入："今天天气真"
语言模型的想法：
  "好"  → 概率 60%
  "差"  → 概率 15%
  "棒"  → 概率 10%
  "热"  → 概率 8%
  其他   → 概率 7%
```

它不是"理解"了这句话，而是从海量文本中**学到了统计规律**：在"今天天气真"后面，"好"出现的频率最高。

> **类比**：语言模型就像一个读了几亿本书的"文字接龙高手"。它不一定真正理解每句话的含义，但它知道什么词最可能出现在什么位置。

2. 什么是"大"语言模型？——参数量的概念

2.1 参数是什么？

想象你在学骑自行车：

- 你的大脑需要调整很多"设定"：身体的平衡角度、踏板的力度、转弯的幅度……
- 每一个"设定"就像一个**参数**

语言模型也一样。它内部有大量的数字（参数），这些数字决定了它如何处理文字。**参数越多，模型就越"聪明"，能记住和处理的知识就越多**。

2.2 图书馆类比

把语言模型想象成一座**图书馆**：

模型	参数量	类比
小型模型	几百万 (M)	一个书架 — 能回答简单问题
MiniMind	6400 万 (64M)	一个小型阅览室 — 能进行基本对话
GPT-2	15 亿 (1.5B)	一层楼的图书馆 — 能写像样的文章
GPT-3	1750 亿 (175B)	整栋国家图书馆 — 几乎无所不知
GPT-4	据传 ~1.8 万亿 (1.8T)	一座城市的所有图书馆 — 多模态超级智能

> **M = Million (百万)，B = Billion (十亿)，T = Trillion (万亿)**

MiniMind 的 64M 参数是 GPT-3 (175B) 的 **1/2700**。这就好比用一个书架的知识去挑战整栋国家图书馆——虽然差距巨大，但正因为"小"，我们才能把它完全拆开学习！

3. GPT 系列的发展历史

GPT 是 **Generative Pre-trained Transformer** 的缩写，意思是"生成式预训练 Transformer"。

时间	模型	参数量	里程碑
2018.06	GPT-1	1.17 亿	证明了"预训练 + 微调"的路线可行
2019.02	GPT-2	15 亿	太强了以至于 OpenAI 一度不敢公开全部权重
2020.06	GPT-3	1750 亿	开创了 In-Context Learning，改变了行业
2022.11	ChatGPT	基于 GPT-3.5	对话式 AI 爆发，AI 进入大众视野
2023.03	GPT-4	未公开 (传 1.8T)	多模态（文字+图片），推理能力大幅提升

核心趋势：模型越来越大、数据越来越多、能力越来越强。但与此同时，训练一次 GPT-4 需要上亿美元——普通人根本玩不起。

这就是 **MiniMind 存在的意义**：用最小的规模复现大模型的完整训练流程，让每个人都能亲手训练一个"大"语言模型。

4. MiniMind 的定位

MiniMind 是一个**开源项目**，它的核心理念是：

> **"用最少的资源，走完大模型训练的全部流程"**

特点	说明
参数量	最小版本仅 64M（约 0.064B）
训练成本	一张消费级 GPU 即可完成全部训练
代码实现	纯 PyTorch 原生实现，不依赖高层封装

特点	说明
训练流程	涵盖预训练 → SFT → RLHF(DPO) 全链路
兼容性	支持 transformers / vLLM / Ollama 等主流框架

MiniMind 就像一辆**教练车**：它不是法拉利，但它拥有完整的发动机、变速箱、刹车系统。通过拆解这辆教练车，你能理解所有跑车的工作原理。

5. LLM 的基本工作原理

大语言模型的工作流程可以用一句话概括：

> **输入一串 token → 模型计算 → 输出下一个 token 的概率分布**

5.1 什么是 Token?

Token 是模型处理文字的**最小单位**。它不完全等于"字"或"词"，而是一种介于两者之间的切分方式。

```
"我喜欢编程" → ["我", "喜欢", "编", "程"]   （4 个 token）
"Hello world" → ["Hello", " world"]          （2 个 token）
```

MiniMind 使用的分词器（Tokenizer）词表大小为 **6400**，也就是说它认识 6400 个不同的 token。相比之下，GPT-4 的词表大约有 10 万个 token。

5.2 从输入到输出的完整流程

```
步骤 1：分词 (Tokenization)
  "今天天气" → [token_1, token_2, token_3, token_4]

步骤 2：转换为数字 (Token IDs)
  [token_1, token_2, token_3, token_4] → [234, 567, 567, 891]

步骤 3：模型计算 (Forward Pass)
  模型内部的数亿个参数一起运算，产生一个概率分布

步骤 4：输出概率分布
  vocab_size = 6400 个词，每个词一个概率
  "好" → 0.35, "很" → 0.20, "不" → 0.12, ...

步骤 5：选择下一个 token
  根据策略选出一个 token（比如选概率最高的"好"）
```

```
步骤 6：把新 token 拼接到输入后面，重复步骤 1-5
  "今天天气好" → 继续预测下一个 token...
```

这个过程叫做**自回归生成（Autoregressive Generation）**：模型每次只生成一个 token，然后把它加到输入末尾，再预测下一个，周而复始，直到生成完整的句子。

6. Next Token Prediction（下一个词预测）

Next Token Prediction 是几乎所有大语言模型的训练目标，也是它们最核心的能力。

训练阶段

模型会阅读海量文本，对于每一段文字，它都要回答同一个问题：

> "给你前面这些词，下一个词是什么？"

```
训练样本："北京是中国的首都"

模型需要学会：
  给定 "北京"          → 预测 "是"
  给定 "北京是"        → 预测 "中国"
  给定 "北京是中国"    → 预测 "的"
  给定 "北京是中国的" → 预测 "首都"
```

每次预测错了，模型就调整自己的参数，让下次预测得更准——这就是**训练**的过程。

为什么这么简单的目标能产生"智能"？

这是大语言模型最令人惊叹的地方。当你要准确预测下一个词时，你不得不**理解**：

- 语法规则（"我吃了"后面应该接名词）
- 常识知识（"水在 100°C 会..."后面应该接"沸腾"）
- 逻辑推理（"如果 A > B, B > C, 那么 A..."后面应该接"> C"）
- 情感分析（"这部电影太好看了，我..."后面大概率是正面表达）

> **类比**：就像一个人如果能完美预测任何书的下一个字，那他一定对语言、知识、逻辑有极深的理解。

7. Temperature 和采样策略

当模型输出了概率分布后，我们要**选一个 token** 作为结果。怎么选？这就涉及**采样策略**。

7.1 贪心采样 (Greedy)

最简单的方法：每次都选概率最高的那个。

```
概率分布: "好"=0.35, "很"=0.20, "不"=0.12, ...
选择: "好" (概率最高)
```

缺点：生成的文本非常"无聊"，缺乏变化，容易陷入重复。

7.2 Temperature（温度）

Temperature 是一个**控制随机性的旋钮**：

- **Temperature = 0**：完全确定，每次都选最高概率（等于贪心）
- **Temperature = 0.7**：适中的随机性，生成比较自然
- **Temperature = 1.0**：标准随机性
- **Temperature > 1.5**：非常随机，可能胡言乱语

> **类比**：Temperature 就像是给模型的"创造力旋钮"。调低了，模型像个严谨的学者，每次都给最"安全"的回答；调高了，模型像个天马行空的艺术家，说出一些出人意料的话。

7.3 Top-p 采样 (Nucleus Sampling)

只从累积概率前 p% 的 token 中采样。比如 top_p=0.9，就是从概率之和达到 90% 的那些 token 中随机选。

```
概率分布: "好"=0.35, "很"=0.20, "不"=0.12, "棒"=0.10, "冷"=0.08, ...
top_p=0.9: 选取 {"好", "很", "不", "棒", "冷", ...} 直到概率和 ≥ 0.9
从这些候选中随机采样
```

在 MiniMind 的推理代码（`eval_llm.py`）中，你会看到 `temperature` 和 `top_p` 这两个参数：

```
# MiniMind 推理代码（eval_llm.py）
generated_ids = model.generate(
    inputs=inputs["input_ids"],
    max_new_tokens=args.max_new_tokens,
    do_sample=True,
    top_p=args.top_p,
```

```
    temperature=args.temperature,
)
```

8. Decoder-Only 架构为什么成为主流

在 Transformer 的世界里，有三种主要架构：

架构	代表模型	适合任务
Encoder-Only	BERT	理解类任务（分类、抽取）
Encoder-Decoder	T5, BART	翻译、摘要等 seq2seq 任务
Decoder-Only	**GPT, LLaMA, MiniMind**	**文本生成、对话**

为什么 Decoder-Only 赢了？

原因 1：简单就是力量

Decoder-Only 架构只有一条路：从左到右，逐个生成 token。不需要 Encoder，不需要 Cross-Attention，代码更简洁，训练更高效。

> **类比**：Encoder-Decoder 就像是先把中文翻译成"思维语言"（Encoder），再从"思维语言"翻译成英文（Decoder）。Decoder-Only 则跳过了中间步骤，直接"想到哪说到哪"。

原因 2：规模效应（Scaling Law）

研究发现，在相同的计算预算下，Decoder-Only 架构随参数量增大而获得的能力提升更稳定。把钱花在扩大一个 Decoder 上，比花在 Encoder + Decoder 上更划算。

原因 3：统一所有任务

通过巧妙的 prompt 设计，Decoder-Only 模型可以处理几乎所有 NLP 任务：

```
翻译：  "请把以下句子翻译成英文：你好 →"
摘要：  "请总结以下文章：[文章内容] → 摘要："
分类：  "以下评论是正面还是负面？评论：这个产品很好 → 正面"
```

MiniMind 采用的正是 **Decoder-Only** 架构，这也是当前绝大多数大语言模型的选择。

9. 大语言模型的局限性

LLM 虽然强大，但有一些重要的局限：

9.1 幻觉问题 (Hallucination)

模型有时会**一本正经地胡说八道**。它会编造不存在的事实、虚构引用来源、给出错误的数学计算结果——但语气非常自信。

> **为什么会幻觉？** 因为模型的目标是"生成看起来最合理的下一个词"，而不是"生成正确的下一个词"。它不查数据库，也不做事实核验，只是在统计规律中选出最可能的续写。

9.2 无法真正"理解"

模型处理的始终是 token 序列的统计关系，它没有真正的世界模型。问它"一个苹果加一个苹果等于几个苹果"，它可能答对，但并不是因为它"理解"了加法。

9.3 知识截止 (Knowledge Cutoff)

模型的知识停留在训练数据的截止时间。训练数据到 2024 年的模型，不知道 2025 年发生了什么。

9.4 上下文窗口限制

模型一次能处理的 token 数量是有限的。MiniMind 的上下文窗口最大为 32768 个 token（配置中的 `max_position_embeddings`），超出部分会被截断。

📂 对应 MiniMind 源码

概念	对应文件	关键代码
模型配置（参数量等）	`model/model_minimind.py`	`class MiniMindConfig` — 定义了 `hidden_size=768`，`num_hidden_layers=8`，`vocab_size=6400` 等
推理/生成过程	`eval_llm.py`	`model.generate(...)` — 自回归生成，使用 `temperature` 和 `top_p`
Tokenizer（分词器）	`model/tokenizer.json`	词表大小 6400 的分词器配置文件

概念	对应文件	关键代码
模型主体（Decoder-Only）	`model/model_minimind.py`	`class MiniMindForCausalLM` — Decoder-Only 因果语言模型

🎤 面试考点

Q1: 什么是 Next Token Prediction?

答：Next Token Prediction 是大语言模型的核心训练目标。给定一段文本的前 n 个 token，模型需要预测第 n+1 个 token 的概率分布。训练时，模型通过大量文本不断调整参数，使得预测的概率分布尽可能接近真实的下一个 token。这个看似简单的目标，迫使模型学习语法、知识、逻辑等多层次的语言能力。在自回归生成阶段，模型循环地将生成的 token 追加到输入中，逐步产出完整的文本。

Q2: 为什么现在的大模型基本都用 Decoder-Only 架构，而不是 Encoder-Decoder?

答：三个主要原因： 1. **架构简洁**：Decoder-Only 去掉了 Encoder 和 Cross-Attention，代码实现更简单，训练和推理效率更高； 2. **Scaling Law 优势**：实验表明，在相同计算预算下，把全部资源投入 Decoder 比分给 Encoder + Decoder 效果更好； 3. **任务统一性**：通过 prompt engineering，Decoder-Only 模型可以用统一的"续写"方式处理翻译、分类、摘要等几乎所有 NLP 任务，而无需针对不同任务设计不同的模型结构。

Q3: 什么是 LLM 的幻觉问题（Hallucination），为什么会产生?

答：幻觉是指大语言模型生成看似合理但事实错误的内容。产生原因在于模型的本质是**统计续写**——它优化的目标是"让输出的 token 序列在统计上最可能出现"，而非"让输出的内容事实正确"。模型没有外部知识库或事实核验机制，只能依赖训练数据中的统计模式。当训练数据中某个方向的信息不足或相互矛盾时，模型可能会"脑补"出不存在的事实。缓解方法包括：RAG（检索增强生成）、RLHF（人类反馈强化学习）、知识蒸馏等。

Q4: Temperature 参数的作用是什么?

答：Temperature 控制模型输出的随机程度。数学上，它作用于 softmax 之前的 logits：`softmax(logits / T)`。温度越低（趋近 0），概率分布越尖锐，模型越倾向于选择概率最高的 token，输出更确定但可能单调；温度越高（>1），概率分布越平坦，低概率的 token 也有机会

被选中，输出更多样但可能偏离主题。实际应用中，代码生成通常用较低温度（0.2-0.4），创意写作用较高温度（0.7-1.0）。

Q5: MiniMind 的参数量是多少，为什么选择这个规模?

答：MiniMind 最小版本（minimind-3）约 64M 参数。选择这个规模的原因是：(1) 足够小，一张消费级 GPU 就能完成全部训练；(2) 足够完整，包含了 Transformer Decoder 的所有核心组件（Attention、FFN、RoPE、GQA 等）；(3) 方便学习，代码全部由纯 PyTorch 原生实现，没有高层封装，可以逐行阅读理解。

✅ 自测题

题目 1（选择题）

语言模型在生成文本时的核心机制是什么?

A. 从数据库中检索最匹配的答案
B. 根据已有的 token 序列，预测下一个 token 的概率分布
C. 将输入翻译成另一种语言后再翻译回来
D. 通过规则引擎匹配预设的回答模板

> ▼ 查看答案
>
> **B**。语言模型的核心是 Next Token Prediction：根据输入的 token 序列，输出下一个 token 的概率分布，然后通过采样策略选择一个 token，循环往复生成完整文本。

题目 2（选择题）

以下关于 Temperature 参数的描述，**错误**的是?

A. Temperature = 0 时，模型输出完全确定
B. Temperature 越高，输出越随机
C. Temperature 只影响训练过程，不影响推理
D. Temperature 作用于 softmax 之前的 logits

> ▼ 查看答案
>
> **C**。Temperature 是在**推理（生成）阶段**起作用的参数，用于控制输出的随机性，与训练过程无关。

题目 3（简答题）

请简要说明 Encoder-Decoder 架构和 Decoder-Only 架构的区别，以及为什么 Decoder-Only 成为大模型的主流选择。

> **▼ 查看答案**
>
> **区别**：Encoder-Decoder 包含两个部分——Encoder 将输入编码为中间表示，Decoder 根据中间表示生成输出，两者之间通过 Cross-Attention 连接。Decoder-Only 只有 Decoder 部分，直接将输入和输出视为同一个序列，从左到右自回归生成。 **为什么 Decoder-Only 成为主流**：(1) 架构更简单，训练推理效率更高；(2) Scaling Law 实验证明，同等预算下 Decoder-Only 扩展效果更好；(3) 通过 prompt 设计，Decoder-Only 能统一处理几乎所有 NLP 任务。

🔮 下一节预告

下一节我们将深入 Transformer 的内部结构——**注意力机制（Attention）** 是如何让模型"看到"整个句子的？**多头注意力**为什么要"多个角度看问题"？Q、K、V 到底是什么？

👉 **L02 - Transformer 全景图：注意力就是一切**